

BLOCKCHAIN **AUDIT REPORT**

Tabi V3 BlockChain



JANUARY 2026

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	8
2.1 Project Info And Contract Address	8
2.2 Summary	9
2.3 Key Findings	10
3. DETAILED DESCRIPTION OF FINDINGS	12
3.1 Block Gas Estimation Bypass Causes Valid Transactions to be Rejected	12
3.2 GaslessDecorator.handleWrapped Can Drop Context Updates From Wrapped Decorators	14
3.3 Unhandled Panics in Optimistic Processing Goroutine	15
3.4 Race Condition in Optimistic Processing Initialization	17
3.5 Pending Transactions Accepted Without Sufficient Balance Verification	19
3.6 Integer Overflow in IncrGasCounter	21
3.7 Missing Panic Recovery in MetricsPanicCallback	23
3.8 Error Propagation in FinalizeBlocker	25
3.9 Incorrect Sender Address Recovery in EVM RPC Responses	26
3.10 Incorrect Gas Price Estimation Due to Hardcoded Congestion Threshold and Wrong Per- centile Parameter	28
3.11 Incorrect Block Gas Used Calculation in eth_getBlockByNumber/Hash	31
3.12 Bloom Filter Incorrectly Skips Blocks When EVM-Only Bloom Cannot Be Retrieved . . .	34
3.13 Zeroed Storage Slots Not Deleted From Chain State	35
3.14 Incorrect transactionIndex Resolution in eth_getTransactionByBlockAndIndex RPCs	36
3.15 Base Fee Calculation in eth_getTransactionByBlock RPC Responses	38
3.16 Incorrect cumulativeGasUsed Calculation Due to Unordered Transient Receipt Storage	40
3.17 Inconsistent Gas Limit Between RPC Responses and GASLIMIT Opcode	42
3.18 Function isOracleTx Always Returns False Due to Empty Switch Statement	45
3.19 Message URL Needs to Be Changed to Tabi	47
3.20 maxFloat32Safe Should Be 16777215	49
3.21 Need to Add More Unit Test Cases	51
4. CONCLUSION	52

5. APPENDIX	53
5.1 Basic Coding Assessment	53
5.1.1 Apply Verification Control	53
5.1.2 Authorization Access Control	53
5.1.3 Forged Transfer Vulnerability	53
5.1.4 Transaction Rollback Attack	54
5.1.5 Transaction Block Stuffing Attack	54
5.1.6 Soft Fail Attack Assessment	54
5.1.7 Hard Fail Attack Assessment	55
5.1.8 Abnormal Memo Assessment	55
5.1.9 Abnormal Resource Consumption	55
5.1.10 Random Number Security	56
5.2 Advanced Code Scrutiny	56
5.2.1 Cryptography Security	56
5.2.2 Account Permission Control	56
5.2.3 Malicious Code Behavior	57
5.2.4 Sensitive Information Disclosure	57
5.2.5 System API	57
6. DISCLAIMER	58
7. REFERENCES	59

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Tabi V3** to review blockchain implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given Blockchain with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of Blockchains from the perspective of proven programming practices.

Category	Assessment Item
P2P Communication Security	• Connection Number Occupation Audit • Eclipse Attack • Packet Size Limit • Node Communication Protocol Security
RPC Interface Security	• RPC Sensitive Interface Permissions • Traditional Web Security • RPC Interface Security

Consensus Mechanism Security	• Design Of Consensus Mechanism • Implementation Of Consensus Verification • Incentive Mechanism Audit
Transaction processing Security	• Transaction Signature Logic • Transaction Verification Logic • Transaction Processing Logic • Transaction Fee Setting • Transaction Replay
Cryptography Security	• Random Number Range And Probability Distribution • Cryptographic Algorithm Implementation/Use
Wallet Module & Account Security Audit	• Private Key / Mnemonic Word Storage Security • Private Key / Mnemonic Word Usage Security • Private key/mnemonic generation algorithm

Others Security Audit	• Database Security • Thread Security • File Permission Security • Historical Vulnerability Security
------------------------------	--

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Tabi V3	15/01/2026 - 09/02/2026	Go

Project Introduction

Tabi's vision is to become the leading Web3 consumer chain, built on underlying architecture that lowers the barriers of entry for developers and users alike, supporting consumer crypto apps in all becoming killer apps of Web3, and bringing about true mass adoption of crypto. To do so, Tabi will be hyper-focused on onboarding Web2 users into the space and on solving the distribution problem that Web3 consumer apps face.

Tabi will become both a thriving ecosystem for apps and users alike, and also the Web3 coordinating mechanism in our attention economy, by directing users' attention productively to uniquely crypto consumer apps.

Official Website

<https://www.tabichain.com/>

Repository

<https://github.com/tabilabs/tabi-v3>

Commit Hash

467397f1242022b332ebf66d3652eca21d2cfb07

2.2 Summary

Severity	Found
CRITICAL	1
HIGH	5
MEDIUM	11
LOW	3
INFO	1



2.3 Key Findings

Severity	Findings Title	Status
CRITICAL	Block Gas Estimation Bypass Causes Valid Transactions to be Rejected	Fixed
HIGH	GaslessDecorator.handleWrapped Can Drop Context Updates From Wrapped Decorators	Fixed
HIGH	Unhandled Panics in Optimistic Processing Goroutine	Fixed
HIGH	Race Condition in Optimistic Processing Initialization	Fixed
HIGH	Pending Transactions Accepted Without Sufficient Balance Verification	Fixed
HIGH	Integer Overflow in IncrGasCounter	Fixed
MEDIUM	Missing Panic Recovery in MetricsPanicCallback	Fixed
MEDIUM	Error Propagation in FinalizeBlocker	Fixed
MEDIUM	Incorrect Sender Address Recovery in EVM RPC Responses	Fixed
MEDIUM	Incorrect Gas Price Estimation Due to Hardcoded Congestion Threshold and Wrong Percentile Parameter	Fixed
MEDIUM	Incorrect Block Gas Used Calculation in eth_getBlockByNumber/Hash	Fixed
MEDIUM	Bloom Filter Incorrectly Skips Blocks When EVM-Only Bloom Cannot Be Retrieved	Fixed
MEDIUM	Zeroed Storage Slots Not Deleted From Chain State	Fixed
MEDIUM	Incorrect transactionIndex Resolution in eth_getTransactionByBlockAndIndex RPCs	Fixed
MEDIUM	Base Fee Calculation in eth_getTransactionByBlock RPC Responses	Fixed
MEDIUM	Incorrect cumulativeGasUsed Calculation Due to Unordered Transient Receipt Storage	Fixed

Severity	Findings Title	Status
MEDIUM	Inconsistent Gas Limit Between RPC Responses and GASLIMIT Opcode	Fixed
LOW	Function isOracleTx Always Returns False Due to Empty Switch Statement	Fixed
LOW	Message URL Needs to Be Changed to Tabi	Fixed
LOW	maxFloat32Safe Should Be 16777215	Fixed
INFO	Need to Add More Unit Test Cases	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Block Gas Estimation Bypass Causes Valid Transactions to be Rejected

SEVERITY:**CRITICAL****STATUS:****Fixed****PATH:**

app/app.go

DESCRIPTION:

The checkTotalBlockGas function incorrectly uses gasEstimate for block gas limit validation even when the gas estimate exceeds gasWanted. GasWanted is the authoritative transaction gas limit, so using an inflated estimate can cause valid transactions to be rejected during proposal processing.

```
// app/app.go - checkTotalBlockGas
// If the gas estimate is set and at least 21k (the minimum gas needed
// for an EVM tx), use the gas estimate.
// Otherwise, use the gas wanted. Typically the gas estimate is set for
// EVM txs and not set for Cosmos txs.
if decodedTx.GetGasEstimate() >= MinGasEVMtx {
    totalGas += decodedTx.GetGasEstimate() // BUG: Uses estimate even if
    > gasWanted
} else {
    totalGas += gasWanted
}
```

IMPACT:

Valid transactions can be rejected during block proposal validation, which can lead to false proposal failures and reduced network throughput.

RECOMMENDATIONS:

Only use gasEstimate when it does not exceed gasWanted.

```
useEstimate := false
```

```
if decodedTx.GetGasEstimate() >= MinGasEVMtx {  
    if decodedTx.GetGasEstimate() <= gasWanted { // FIX: Only use  
        estimate if <= gasWanted  
        useEstimate = true  
    }  
}  
if useEstimate {  
    totalGas += decodedTx.GetGasEstimate()  
} else {  
    totalGas += gasWanted  
}
```

3.2 GaslessDecorator.handleWrapped Can Drop Context Updates From Wrapped Decorators

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**`app/antedecorators/gasless.go`**DESCRIPTION:**

GaslessDecorator.handleWrapped may fail to propagate the sdk.Context returned by each wrapped decorator's AnteHandle. In the Cosmos SDK, sdk.Context is effectively immutable and decorators apply changes by returning a new context. If the loop does not update ctx with the returned context, subsequent decorators and the final next(...) handler can run with a stale context.

IMPACT:

Critical ante-chain changes (gas meter updates, priority changes, event manager updates, and context values) can be silently dropped, leading to subtle and hard-to-debug behavior differences.

RECOMMENDATIONS:

Capture and chain the returned context from each wrapped decorator exactly once.

```
for _, handler := range gd.wrapped {
    newCtx, err := handler.AnteHandle(ctx, tx, simulate,
    terminatorHandler)
    if err != nil {
        return newCtx, err
    }
    ctx = newCtx
}
return next(ctx, tx, simulate)
```

3.3 Unhandled Panics in Optimistic Processing Goroutine

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

app/abci.go

DESCRIPTION:

ProcessProposalHandler spawns a goroutine for optimistic processing that runs ProcessBlock asynchronously without panic recovery. If a transaction causes a panic during processing (for example, malformed data causing GetSigners() to panic), the goroutine can die silently and the chain can halt.

```
go func() {
    events, txResults, endBlockResp, _ := app.ProcessBlock(ctx, req.Txs,
        req, req.ProposedLastCommit, false)
    // Store results...
}()
```

IMPACT:

A single malicious transaction can halt the blockchain network, resulting in a denial of service.

RECOMMENDATIONS:

Add panic recovery that catches all panics, returns errors instead of crashing, and preserves upgrade-related panics.

```
func (app *App) ProcessBlock(...) (events []abci.Event, txResults
[]*abci.ExecTxResult, endBlockResp abci.ResponseEndBlock, err error) {
    defer func() {
        if r := recover(); r != nil {
            panicMsg := fmt.Sprintf('%v', r)
            // Re-panic for upgrade-related panics to allow proper
            upgrade mechanism
            if upgradePanicRe.MatchString(panicMsg) {
```

```
        ctx.Logger().Error('upgrade panic detected, panicking to
trigger upgrade', 'panic', r)
        panic(r) // Re-panic to trigger upgrade mechanism
    }
    ctx.Logger().Error('panic recovered in ProcessBlock', '
panic', r)
    err = fmt.Errorf('ProcessBlock panic: %v', r)
    events = nil
    txResults = nil
    endBlockResp = abci.ResponseEndBlock{}
    }
    }()
    // ... processing logic
}
```


3.4 Race Condition in Optimistic Processing Initialization

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

app/app.go

DESCRIPTION:

The optimistic processing initialization uses a check-then-act pattern that reads Completion outside of the mutex. Multiple goroutines can observe Completion == nil before acquiring the lock, leading to multiple optimistic processing goroutines being started.

```
// app/app.go
// Before: Read outside lock
if app.GetOptimisticProcessingInfo().Completion == nil {
    completionSignal := make(chan struct{}, 1)
    app.optimisticProcessingInfoMutex.Lock()
    app.optimisticProcessingInfo = OptimisticProcessingInfo{ ... }
    app.optimisticProcessingInfoMutex.Unlock()
}
```

IMPACT:

Multiple optimistic processing goroutines can start, wasting resources and duplicating work for the same block.

RECOMMENDATIONS:

Check and write within the same lock to avoid the race.

```
app.optimisticProcessingInfoMutex.Lock()
shouldStartOptimisticProcessing :=
    app.optimisticProcessingInfo.Completion == nil
if shouldStartOptimisticProcessing {
    app.optimisticProcessingInfo = OptimisticProcessingInfo{ ... }
}
```

```
app.optimisticProcessingInfoMutex.Unlock()  
if shouldStartOptimisticProcessing {  
    // Start goroutine  
}
```

3.5 Pending Transactions Accepted Without Sufficient Balance Verification

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

x/evm/ante/sig.go

DESCRIPTION:

EVMSigVerifyDecorator.AnteHandle does not verify that the sender still has sufficient funds when processing pending EVM transactions. When a transaction's nonce is within the pending range, the code accepts it without re-checking balance.

```
} else if txNonce < nextPendingNonce {  
    // this nonce is allowed to process as it is part of the  
    // consecutive nonces from nextNonceToBeMined to nextPendingNonce  
    // This logic allows multiple nonces from an account to be processed  
    // in a block.  
    return abci.Accepted  
}
```

IMPACT:

Transactions can pass ante-handler validation but fail during execution when balance is checked, wasting validator compute resources and confusing users.

RECOMMENDATIONS:

Re-check balance for pending transactions and keep them in the pending pool if funds are insufficient.

```
} else if txNonce < nextPendingNonce {  
    // check if the sender still has enough funds to pay for gas  
    balance := svd.evmKeeper.BankKeeper().GetBalance(latestCtx,  
types.MustGetEVMTransactionMessage(tx).Derived.SenderTabiAddr,  
evmkeeper.BaseDenom).Amount  
    if balance.LT(sdk.NewIntFromBigInt(fee)) {
```

```
        // not enough funds. Go back to pending as it may obtain
        // sufficient funds later.
        return abci.Pending
    }
    // this nonce is allowed to process as it is part of the
    // consecutive nonces from nextNonceToBeMined to nextPendingNonce
}

fee := new(big.Int).Mul(ethTx.GasPrice(),
    new(big.Int).SetUint64(ethTx.Gas()))
if ethTx.Value() != nil {
    fee = new(big.Int).Add(fee, ethTx.Value())
}
```

3.6 Integer Overflow in IncrGasCounter

SEVERITY:**HIGH****STATUS:****Fixed****PATH:**

app/utils.go

DESCRIPTION:

IncrGasCounter casts an int64 gas value directly to float32. float32 can only safely represent integers up to 2^{24} (16,777,215), while gas values can be much larger. This can cause precision loss or overflow.

```
func IncrGasCounter(gasType string, value int64) {
    SafeTelemetryIncrCounterWithLabels(
        []string{'tabi', 'tx', 'gas', 'counter'},
        float32(value), // VULNERABILITY: Direct cast without bounds
        checking
        []metrics.Label{telemetry.NewLabel('type', gasType)},
    )
}
```

IMPACT:

Telemetry counters can become inaccurate and may wrap or lose precision for large gas values.

RECOMMENDATIONS:

Add overflow protection and cap values above the float32 safe range.

```
func IncrGasCounter(gasType string, value int64) {
    const maxSafeFloat32 = 1<<24 - 1 // Maximum safe integer
    representable in float32 (16,777,215)

    if value < 0 {
        // Log negative values but don't panic - this shouldn't happen in
        normal operation
    }
}
```

```
telemetry.IncrCounterWithLabels(  
    []string{ 'tabi', 'tx', 'gas', 'counter', 'error' },  
    1,  
    []metrics.Label{  
        telemetry.NewLabel('type', gasType),  
        telemetry.NewLabel('error', 'negative_value'),  
    },  
)  
return  
}  
  
if value > maxSafeFloat32 {  
    // Cap the value to prevent overflow while logging the incident  
    telemetry.IncrCounterWithLabels(  
        []string{ 'tabi', 'tx', 'gas', 'counter', 'overflow' },  
        1,  
        []metrics.Label{  
            telemetry.NewLabel('type', gasType),  
            telemetry.NewLabel('original_value', 'capped'),  
        },  
    )  
    value = maxSafeFloat32  
}  
  
SafeTelemetryIncrCounterWithLabels(  
    []string{ 'tabi', 'tx', 'gas', 'counter' },  
    float32(value),  
    []metrics.Label{telemetry.NewLabel('type', gasType)},  
)  
}
```

3.7 Missing Panic Recovery in MetricsPanicCallback

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

utils/panic.go

DESCRIPTION:

MetricsPanicCallback handles and logs panics, but it has no internal recovery. If the telemetry or logging operations panic, the error propagates and can crash the node.

```
// utils/panic.go
func MetricsPanicCallback(err any, ctx sdk.Context, key string) {
    ctx.Logger().Error(fmt.Sprintf('panic %s occurred during order
    matching for: %s', err, key))
    telemetry.IncrCounterWithLabels(
        []string{'panic'},
        1,
        []metrics.Label{
            telemetry.NewLabel('error', fmt.Sprintf('%s', err)),
            telemetry.NewLabel('module', key),
        },
    )
}
```

IMPACT:

A node can crash during panic handling, preventing graceful error recovery.

RECOMMENDATIONS:

Add a defer/recover block at the top of the function to catch any panics during logging or metrics recording.

```
func MetricsPanicCallback(err any, ctx sdk.Context, key string) {
    defer func() {
```

```
        if r := recover(); r != nil {
            ctx.Logger().Error('panic recovered in MetricsPanicCallback',
                'panic', r)
        }
    }()
    ctx.Logger().Error(fmt.Sprintf('panic %s occurred during order
    matching for: %s', err, key))
    telemetry.IncrCounterWithLabels(
        []string{'panic'},
        1,
        []metrics.Label{
            telemetry.NewLabel('error', fmt.Sprintf('%s', err)),
            telemetry.NewLabel('module', key),
        },
    )
}
```


3.8 Error Propagation in FinalizeBlocker

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

app/app.go

DESCRIPTION:

Errors from ProcessBlock are ignored in FinalizeBlocker, which hides failures and can lead to inconsistent state.

```
// Before:
events, txResults, endBlockResp, _ := app.ProcessBlock(...) // Error
ignored!
```

IMPACT:

Failures in block processing can go unnoticed while execution continues with partial results.

RECOMMENDATIONS:

Handle and return errors from ProcessBlock.

```
events, txResults, endBlockResp, processErr := app.ProcessBlock(...)
if processErr != nil {
    ctx.Logger().Error('ProcessBlock failed in FinalizeBlocker', 'error',
processErr)
    return nil, processErr
}
```

3.9 Incorrect Sender Address Recovery in EVM RPC Responses

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

evmrpc/tx.go

DESCRIPTION:

EVM RPC endpoints (for example, `eth_getTransactionByHash` and `eth_getTransactionReceipt`) recover sender addresses using a signer derived from the latest block context, not the transaction's block context. This yields incorrect signer versions and incorrect sender addresses for some transactions, especially legacy chain ID 0 transactions.

```
// evmrpc/tx.go - GetTransactionByHash
sdkCtx := t.ctxProvider(LatestCtxHeight) // BUG: Uses latest context
signer := ethtypes.MakeSigner(
    types.DefaultChainConfig().EthereumConfig(t.keeper.ChainID(sdkCtx)),
    big.NewInt(sdkCtx.BlockHeight()), // BUG: Latest block, not tx's
    block
    uint64(sdkCtx.BlockTime().Unix()), // BUG: Latest time, not tx's
    time
)
from, _ := ethtypes.Sender(signer, etx) // BUG: Returns wrong address
for chain ID 0
```

IMPACT:

RPC responses can include incorrect from addresses, breaking API compatibility and client assumptions.

RECOMMENDATIONS:

Recover the sender using the transaction's actual block height/time and handle chain ID 0 correctly.

```
// evmrpc/rpcutils/sig.go - New correct implementation
```

```
func RecoverEVMSender(ethTx *ethtypes.Transaction, blockHeight int64,
    blockTime int64) (common.Address, error) {
    chainID := ethTx.ChainId()

    var signer ethtypes.Signer
    if chainID.Int64() == 0 {
        signer = ethtypes.NewEIP155Signer(chainID) // Handles chain ID 0
    } else {
        version := getSignerVersion(blockHeight, uintBlockTime, ethCfg)
        // Uses tx's block
        signer = signerMap[version](chainID)
    }
    // ... proper V adjustment and hash computation
}
```

3.10 Incorrect Gas Price Estimation Due to Hardcoded Congestion Threshold and Wrong Percentile Parameter

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

evmrpc/info.go

DESCRIPTION:

The `eth_gasPrice` and `eth_maxPriorityFeePerGas` RPC endpoints use a hardcoded congestion threshold and pass the wrong percentile value to `FeeHistory`. The code uses 0.5 (0.5th percentile) instead of 50 (median), and a fixed 8.5M gas threshold that does not reflect the actual block gas limit.

```
// evmrpc/info.go
const highTotalGasUsedThreshold = 8500000 // BUG: Hardcoded, ignores
    actual gas limit

func (i *InfoAPI) GasPrice(ctx context.Context) (*hexutil.Big, error) {
    // ...
    feeHist, err := i.FeeHistory(ctx, 1, rpc.LatestBlockNumber,
    []float64{0.5}) // BUG: 0.5 not 50
    // ...
}

func (i *InfoAPI) GasPriceHelper(...) (*hexutil.Big, error) {
    isChainCongested := totalGasUsedPrevBlock > highTotalGasUsedThreshold
    // BUG: Hardcoded threshold
    // ...
}

func (i *InfoAPI) MaxPriorityFeePerGas(ctx context.Context)
(*hexutil.Big, error) {
    // ...
    isChainCongested := totalGasUsed > highTotalGasUsedThreshold // BUG:
    Hardcoded threshold
    // ...
    feeHist, err := i.FeeHistory(ctx, 1, rpc.LatestBlockNumber,
    []float64{0.5}) // BUG: 0.5 not 50
```

```
// ...  
}
```

IMPACT:

Gas price recommendations and priority fee estimates are incorrect for both `eth_gasPrice` and `eth_maxPriorityFeePerGas`.

RECOMMENDATIONS:

Use a percentile value of 50 and compute congestion thresholds based on the actual block gas limit.

```
// evmrpc/info.go  
const DefaultBlockGasLimit = 10000000  
const defaultThresholdPercentage = 80 // 80%  
  
func (i *InfoAPI) GasPrice(ctx context.Context) (*hexutil.Big, error) {  
    // ...  
    feeHist, err := i.FeeHistory(ctx, 1, rpc.LatestBlockNumber,  
    []float64{50}) // FIX: 50 for median  
    // ...  
}  
  
func (i *InfoAPI) GasPriceHelper(...) (*hexutil.Big, error) {  
    isChainCongested := i.isChainCongested(totalGasUsedPrevBlock) //  
    FIX: Dynamic threshold  
    // ...  
}  
  
// FIX: New function calculates threshold based on actual block gas limit  
func (i *InfoAPI) isChainCongested(totalGasUsed uint64) bool {  
    sdkCtx := i.ctxProvider(LatestCtxHeight)  
    var gasLimit uint64  
    if sdkCtx.ConsensusParams() != nil && sdkCtx.ConsensusParams().Block  
    != nil {  
        maxGas := sdkCtx.ConsensusParams().Block.MaxGas  
        if maxGas <= 0 {  
            gasLimit = uint64(DefaultBlockGasLimit)  
        } else {  
            gasLimit = uint64(maxGas)  
        }  
    }  
}
```

```
    } else {  
        gasLimit = uint64(DefaultBlockGasLimit)  
    }  
    threshold := gasLimit * uint64(defaultThresholdPercentage) / 100  
    return totalGasUsed > threshold  
}
```

3.11 Incorrect Block Gas Used Calculation in eth_getBlockByNumber/Hash

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

evmrpc/block.go

DESCRIPTION:

eth_getBlockByNumber and eth_getBlockByHash return incorrect gasUsed values because blockGasUsed is accumulated before verifying whether a transaction is included in the response. Gas from skipped transactions is still counted.

```
// evmrpc/block.go - EncodeTmBlock
for i, txRes := range blockRes.TxsResults {
    blockGasUsed += txRes.GasUsed // BUG: Adds gas BEFORE checking if tx
    is included
    decoded, err := txDecoder(block.Block.Txs[i])
    if err != nil {
        return nil, errors.New('failed to decode transaction')
    }
    for _, msg := range decoded.GetMsgs() {
        switch m := msg.(type) {
        case *types.MsgEVMTransaction:
            if m.IsAssociateTx() {
                continue // Skipped, but gas was already added!
            }
            // ... more skip conditions ...
            if !fullTx {
                transactions = append(transactions, hash)
            } else {
                // ...
            }
        }
    }
}
```

IMPACT:

RPC block responses report incorrect gasUsed values, violating Ethereum JSON-RPC expectations.

RECOMMENDATIONS:

Only add gas for transactions that are actually included in the response.

```
// evmrpc/block.go - EncodeTmBlock
for i, txRes := range blockRes.TxsResults {
    // REMOVED: blockGasUsed += txRes.GasUsed
    decoded, err := txDecoder(block.Block.Txs[i])
    if err != nil {
        return nil, errors.New('failed to decode transaction')
    }
    for _, msg := range decoded.GetMsgs() {
        switch m := msg.(type) {
        case *types.MsgEVMTransaction:
            if m.IsAssociateTx() {
                continue
            }
            // ... skip conditions ...
            receipt, err := k.GetReceipt(ctx, hash)
            // ...
            if !fullTx {
                blockGasUsed += int64(receipt.GasUsed) // FIX: Add gas
                only when tx is included
                transactions = append(transactions, hash)
            } else {
                blockGasUsed += int64(receipt.GasUsed) // FIX: Add gas
                only when tx is included
                newTx := export.NewRPCTransaction(...)
                transactions = append(transactions, newTx)
            }
        case *wasmtypes.MsgExecuteContract:
            // Similar fix for WASM transactions
            if !fullTx {
                blockGasUsed += int64(receipt.GasUsed)
                transactions = append(transactions, ...)
            } else {
                blockGasUsed += int64(receipt.GasUsed)
                transactions = append(transactions, rpcTx)
            }
        default:
            // Bank transfers use txRes.GasUsed
        }
    }
}
```



```
        if !fullTx {
            blockGasUsed += txRes.GasUsed
            transactions = append(transactions, ...)
        } else {
            blockGasUsed += txRes.GasUsed
            transactions = append(transactions, rpcTx)
        }
    }
}
```

3.12 Bloom Filter Incorrectly Skips Blocks When EVM-Only Bloom Cannot Be Retrieved

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

evmrpc/filter.go

DESCRIPTION:

LogFetcher.processBatch skips blocks when the bloom filter does not match, but it does not guard against empty or unavailable bloom data. If GetEvmOnlyBlockBloom returns an all-zero bloom, the block is incorrectly skipped.

```
// Vulnerable code - no guard against empty bloom
if !MatchFilters(blockBloom, bloomIndexes) {
    <-dbReadSemaphore
    continue // skip the block if bloom filter does not match
}
```

IMPACT:

Logs can be silently omitted from eth_getLogs results, leading to incomplete indexing and incorrect application state.

RECOMMENDATIONS:

Only apply bloom filtering when valid bloom data exists.

```
// only apply bloom filter optimization when valid data exists
if blockBloom != (ethtypes.Bloom{}) && !MatchFilters(blockBloom,
    bloomIndexes) {
    <-dbReadSemaphore
    continue // skip the block if bloom filter does not match
}
```

3.13 Zeroed Storage Slots Not Deleted From Chain State

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`x/evm/keeper/state.go`**DESCRIPTION:**

SetState does not delete storage slots when they are set to zero. Instead, it stores the zero value, causing cleared slots to persist in chain state.

```
func (k *Keeper) SetState(ctx sdk.Context, addr common.Address, key
    common.Hash, val common.Hash) {
    k.PrefixStore(ctx, types.StateKey(addr)).Set(key[:], val[:])
}
```

IMPACT:

Chain state grows monotonically even when contracts delete data, increasing storage requirements and sync costs over time.

RECOMMENDATIONS:

Delete storage slots when the value is zero.

```
func (k *Keeper) SetState(ctx sdk.Context, addr common.Address, key
    common.Hash, val common.Hash) {
    store := k.PrefixStore(ctx, types.StateKey(addr))
    if val == (common.Hash{}) {
        store.Delete(key[:])
        return
    }
    store.Set(key[:], val[:])
}
```

3.14 Incorrect transactionIndex Resolution in eth_getTransactionByBlockAndIndex RPCs

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

evmrpc/tx.go

DESCRIPTION:

eth_getTransactionByBlockNumberAndIndex and eth_getTransactionByBlockHashAndIndex interpret the caller-supplied transactionIndex as a Cosmos block index instead of an EVM-only index. This causes the wrong transaction to be returned when Cosmos and EVM transactions are mixed in a block.

```
// evmrpc/tx.go - GetTransactionByBlockNumberAndIndex
func (t *TransactionAPI) GetTransactionByBlockNumberAndIndex(ctx
    context.Context, blockNr rpc.BlockNumber, index hexutil.Uint)
    (*export.RPCTransaction, error) {
    ...
    return t.getTransactionWithBlock(block, index, t.includeSynthetic)
    // index used as Cosmos index
}

// getTransactionWithBlock
func (t *TransactionAPI) getTransactionWithBlock(block
    *coretypes.ResultBlock, index hexutil.Uint, ...)
    (*export.RPCTransaction, error) {
    if int(index) >= len(block.Block.Txs) {
        return nil, nil
    }
    txIdx, found, ethTx, _ := GetEvmTxIndex(block.Block.Txs,
        uint32(index), ...)
    ...
}
```

IMPACT:

Callers querying by EVM index receive the wrong transaction, and the API becomes inconsistent with Ethereum JSON-RPC semantics.

RECOMMENDATIONS:

Introduce a TransactionIndex abstraction that translates EVM indices to Cosmos indices before accessing `block.Block.Txs`.

3.15 Base Fee Calculation in eth_getTransactionByBlock RPC Responses

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

evmrpc/tx.go

DESCRIPTION:

getTransactionWithBlock retrieves baseFeePerGas from the current block context, but EIP-1559 transactions require the base fee from the previous block to compute effective gas price. The edge case for block height 1 is also not handled.

```
func (t *TransactionAPI) getTransactionWithBlock(block
    *coretypes.ResultBlock, index hexutil.Uint, includeSynthetic bool)
(*export.RPCTransaction, error) {
    // ... transaction lookup ...

    height := int64(receipt.BlockNumber)
    baseFeePerGas := t.keeper.GetBaseFee(t.ctxProvider(height)) //
    WRONG: uses current block

    chainConfig :=
types.DefaultChainConfig().EthereumConfig(t.keeper.ChainID(t.ctxProvider(height)))
    blockHash := common.HexToHash(block.BlockID.Hash.String())
    blockNumber := uint64(block.Block.Height)
    blockTime := block.Block.Time
    res := export.NewRPCTransaction(ethTx, blockHash, blockNumber,
    uint64(blockTime.Second()), uint64(txIdx), baseFeePerGas, chainConfig)
    return res, nil
}
```

IMPACT:

The gasPrice field in transaction responses is incorrect for EIP-1559 transactions.

RECOMMENDATIONS:

Retrieve the next base fee from the previous block (height - 1) and handle block height 1 explicitly.

```
height := int64(receipt.BlockNumber)
var baseFeePerGas *big.Int
if block.Block.Height > 1 {
    baseFeePerGas = t.keeper.GetNextBaseFeePerGas(t.ctxProvider(height -
    1)).TruncateInt().BigInt()
} else {
    baseFeePerGas = types.DefaultMinFeePerGas.TruncateInt().BigInt()
}
```

3.16 Incorrect cumulativeGasUsed Calculation Due to Unordered Transient Receipt Storage

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**

x/evm/keeper/receipt.go

DESCRIPTION:

The transient receipt store uses only the transaction hash as the key, which means receipts are iterated in hash order rather than transaction order. This breaks cumulativeGasUsed calculation when flushing receipts.

```
func (k *Keeper) SetTransientReceipt(ctx sdk.Context, txHash common.Hash,
    receipt *types.Receipt) error {
    // ...
    store.Set(types.ReceiptKey(txHash), bz) // Key: ReceiptKeyPrefix +
    txHash
    return nil
}

func (k *Keeper) flushTransientReceipts(ctx sdk.Context, sync bool) error
{
    iter := prefix.NewStore(ctx.TransientStore(k.transientStoreKey),
    types.ReceiptKeyPrefix).Iterator(nil, nil)
    defer iter.Close()
    var pairs []*iavl.KVPair
    for ; iter.Valid(); iter.Next() {
        // Iterates in hash order, NOT transaction order
        // Cannot calculate cumulative gas correctly
        kvPair := &iavl.KVPair{Key:
        types.ReceiptKey(common.Hash(iter.Key())), Value: iter.Value()}
        pairs = append(pairs, kvPair)
    }
    // ...
}
```


IMPACT:

The cumulativeGasUsed field in receipts is 0 or incorrect, violating the Ethereum JSON-RPC specification.

RECOMMENDATIONS:

Include the transaction index in the transient receipt key and iterate receipts in transaction order.

```
type TransientReceiptKey []byte

func NewTransientReceiptKey(txIndex uint64, txHash common.Hash)
    TransientReceiptKey {
    return append(ReceiptKeyPrefix, fmt.Sprintf('%020d:%s', txIndex,
        txHash.String()))[:...]
}

func (trk TransientReceiptKey) TransactionHash() common.Hash {
    if i := bytes.LastIndexByte(trk, ':'); i != -1 {
        return common.HexToHash(string(trk[i+1:]))
    }
    return common.Hash{}
}

store.Set(types.NewTransientReceiptKey(uint64(receipt.TransactionIndex),
    txHash), bz)

func (k *Keeper) flushTransientReceipts(ctx sdk.Context, sync bool) error
{
    iter := transientReceiptStore.Iterator(nil, nil) // Now iterates in
    txIndex order
    cumulativeGasUsedPerBlock := make(map[uint64]uint64)
    for ; iter.Valid(); iter.Next() {
        receipt := &types.Receipt{}
        receipt.Unmarshal(iter.Value())
        cumulativeGasUsedPerBlock[receipt.BlockNumber] += receipt.GasUsed
        receipt.CumulativeGasUsed =
        cumulativeGasUsedPerBlock[receipt.BlockNumber]
        // Marshal and store with correct cumulative gas
    }
}
```

3.17 Inconsistent Gas Limit Between RPC Responses and GASLIMIT Opcode

SEVERITY:**MEDIUM****STATUS:****Fixed****PATH:**`x/evm/keeper/keeper.go`**DESCRIPTION:**

The EVM BlockContext.GasLimit is set to the transaction gas pool (`gp.Gas()`) instead of the block's consensus gas limit. RPC headers also use a configurable gas cap instead of the consensus gas limit, causing mismatches between opcode results and RPC responses.

```
func (k *Keeper) GetVMBlockContext(ctx sdk.Context, gp core.GasPool)
(*vm.BlockContext, error) {
    // ...
    return &vm.BlockContext{
        // ...
        GasLimit:    gp.Gas(), // WRONG: uses transaction gas pool, not
        block gas limit
        // ...
    }, nil
}

header := &ethtypes.Header{
    // ...
    GasLimit:    b.config.GasCap, // Uses RPC config, not consensus
    params
    // ...
}
```

IMPACT:

Contracts that rely on `block.gaslimit` receive incorrect values, and EVM behavior diverges from the Ethereum specification.

RECOMMENDATIONS:

Use consensus parameters for gas limit and define a safe default when they are unavailable.

```
func (k *Keeper) GetVMBlockContext(ctx sdk.Context, gp core.GasPool)
(*vm.BlockContext, error) {
    // ...
    return &vm.BlockContext{
        // ...
        GasLimit: func() uint64 {
            if ctx.ConsensusParams() != nil &&
ctx.ConsensusParams().Block != nil {
                return uint64(ctx.ConsensusParams().Block.MaxGas)
            }
            return DefaultBlockGasLimit // Define a reasonable default
(e.g., 10000000)
        }(),
        // ...
    }, nil
}

func (b *Backend) getHeader(blockNumber *big.Int) *ethtypes.Header {
    // Get block results to access consensus parameters
    number := blockNumber.Int64()
    block, blockErr := blockByNumber(context.Background(), b.tmClient,
&number)
    var gasLimit uint64
    if blockErr == nil {
        blockRes, blockResErr :=
blockResultsWithRetry(context.Background(), b.tmClient, &number)
        if blockResErr == nil && blockRes.ConsensusParamUpdates != nil &&
blockRes.ConsensusParamUpdates.Block != nil {
            gasLimit = uint64(blockRes.ConsensusParamUpdates.Block.MaxGas)
        } else {
            gasLimit = keeper.DefaultBlockGasLimit
        }
    } else {
        gasLimit = keeper.DefaultBlockGasLimit
    }

    header := &ethtypes.Header{
        // ...
        GasLimit: gasLimit, // Use actual consensus gas limit
        // ...
    }
}
```

}

3.18 Function isOracleTx Always Returns False Due to Empty Switch Statement

SEVERITY:

LOW

STATUS:

Fixed

PATH:

app/antdecorators/priority.go

DESCRIPTION:

The isOracleTx function contains a switch statement with no case clauses, only a default branch that returns false. This makes the function always return false, which renders OraclePriority assignment in AnteHandle unreachable.

```
func isOracleTx(tx sdk.Tx) bool {
    if len(tx.GetMsgs()) == 0 {
        // empty TX isn't oracle
        return false
    }
    for _, msg := range tx.GetMsgs() {
        switch msg.(type) {
        default:
            return false
        }
    }
    return true
}
```

IMPACT:

Oracle priority assignment is never applied, which can impact intended transaction prioritization.

RECOMMENDATIONS:

Return false explicitly or remove the function to avoid misleading logic.

```
func isOracleTx(_ sdk.Tx) bool {
    return false
}
```

}

3.19 Message URL Needs to Be Changed to Tabi

SEVERITY:

LOW

STATUS:

Fixed

PATH:

utils/prioritized_txs.go

DESCRIPTION:

Some oracle message type URLs still reference Sei. Tabi naming should be used instead for consistency.

```
var oracleMsgTypeURLs = map[string]struct{}{
    // Sei / Oracle module message types
    '/seiprotocol.sei.oracle.MsgAggregateExchangeRatePrevote' : {},
    '/seiprotocol.sei.oracle.MsgAggregateExchangeRateVote' :    {},
    '/seiprotocol.sei.oracle.MsgDelegateFeedConsent' :          {},
    // General oracle module message types
    '/oracle.MsgAggregateExchangeRatePrevote' : {},
    '/oracle.MsgAggregateExchangeRateVote' :    {},
    '/oracle.MsgDelegateFeedConsent' :          {},
}
```

IMPACT:

Inconsistent message naming can cause confusion in documentation and audit trails and can mislead readers about the expected message types.

RECOMMENDATIONS:

Replace Sei oracle URLs with Tabi equivalents.

```
var oracleMsgTypeURLs = map[string]struct{}{
-   // Sei / Oracle module message types
-   '/seiprotocol.sei.oracle.MsgAggregateExchangeRatePrevote' : {},
-   '/seiprotocol.sei.oracle.MsgAggregateExchangeRateVote' :    {},
-   '/seiprotocol.sei.oracle.MsgDelegateFeedConsent' :          {},
}
```

```
+ // Tabi oracle module message types
+ '/tabilabs.tabi.oracle.MsgAggregateExchangeRatePrevote' ': {}',
+ '/tabilabs.tabi.oracle.MsgAggregateExchangeRateVote' ': {}',
+ '/tabilabs.tabi.oracle.MsgDelegateFeedConsent' ': {}',
  // General oracle module message types
  '/oracle.MsgAggregateExchangeRatePrevote' ': {}',
  '/oracle.MsgAggregateExchangeRateVote' ': {}',
  '/oracle.MsgDelegateFeedConsent' ': {}',
}
```


3.20 maxFloat32Safe Should Be 16777215

SEVERITY:

LOW

STATUS:

Fixed

PATH:

utils/metrics/metrics_util.go

DESCRIPTION:

The IncrGasCounter function sets maxFloat32Safe to 16777216 (2^{24}), which is the point where float32 loses unit resolution rather than a safe upper bound. It also caps iterations at maxIterations = 100, which limits the maximum range processed per call.

```
func IncrGasCounter(gasType string, value int64) {
    if value <= 0 {
        return
    }
    const maxFloat32Safe int64 = 16777216
    const maxIterations = 100
    for i := 0; value > 0 && i < maxIterations; i++ {
        incr := value
        if incr > maxFloat32Safe {
            incr = maxFloat32Safe
        }
        telemetry.IncrCounterWithLabels(
            []string{'tabi', 'tx', 'gas', 'counter'},
            float32(incr),
            []metrics.Label{telemetry.NewLabel('type', gasType)},
        )
        value -= incr
    }
}
```

IMPACT:

This logic can lead to severe distortion of monitoring data when the volume is large.

RECOMMENDATIONS:

Use 16777215 instead.

```
func IncrGasCounter(gasType string, value int64) {
    if value <= 0 {
        return
    }
-   const maxFloat32Safe int64 = 16777216
+   const maxFloat32Safe int64 = 16777215
    const maxIterations = 100
    for i := 0; value > 0 && i < maxIterations; i++ {
        incr := value
        if incr > maxFloat32Safe {
            incr = maxFloat32Safe
        }
        telemetry.IncrCounterWithLabels(
            []string{'tabi', 'tx', 'gas', 'counter'},
            float32(incr),
            []metrics.Label{telemetry.NewLabel('type', gasType)},
        )
        value -= incr
    }
}
```

3.21 Need to Add More Unit Test Cases

SEVERITY:

INFO

STATUS:

Fixed

PATH:**DESCRIPTION:**

Recommended New Tests:

1. `utils/prioritized_txs_test.go`
 - Test `IsTxPrioritized` with oracle messages
 - Test `IsTxPrioritized` with non-oracle messages
 - Test `IsTxPrioritized` with nil tx
 - Test `isOracleMsg` with various type URLs
 - Test `containsOracleInAuthz` with nested authz up to depth limit
 - Test authz depth limiting (>5 levels returns false)
2. `app/antedecorators/gasless_test.go`
 - Test context propagation through multiple wrapped handlers

IMPACT:

Insufficient test coverage increases regression risk and reduces confidence in transaction prioritization and gasless decorator behavior.

RECOMMENDATIONS:

Add the unit tests listed above to improve coverage and prevent regressions.

4. CONCLUSION

In this audit, we thoroughly analyzed **Tabi V3** blockchain implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec